DICA DO PROJETO

Aumentando seu repertório

Dando vida ao *chatbot*

Chegou o momento mais esperado: transformar nosso *chatbot* de "eco" em uma IA de verdade.

Até agora, quando o usuário perguntava algo, a resposta era apenas uma repetição ou variação do que foi perguntado. A partir desta semana, vamos conectá-lo a uma LLM real.

Mas antes de escrevermos código, gostaria de lembrá-lo que temos duas opções:

Opção	Custo	Velocidade	Privacidade	Recomendação
LLMs locais (Ollama/LM Studio)	Gratuito	Depende do seu computador	Total (dados ficam no seu computador)	Recomendado para aprender
APIs na nuvem (OpenAI etc.)	Pago por uso	Muito rápida	Dados vão para nuvem	Recomendado para uma solução real

A autor.

Esta disciplina foi projetada para você completar **todo o projeto sem gastar um centavo**.

A opção local é tão boa quanto as APIs pagas para aprendizado. Contudo, se você quiser pode usar uma API paga, sem problema algum. Apenas peço a você responsabilidade e cautela no monitoramento de custos.

#FICADICA

Você pode usar o Ollama ou qualquer alternativa local no lugar do LM Studio.

APIs na nuvem

Se você preferir usar modelos mais poderosos e não se importar com custos, pode usar APIs de provedores como OpenAI, Google ou Anthropic.

O custo depende da versão do modelo e do fornecedor. Geralmente, modelos mais novos serão mais caros do que os mais antigos, e modelos mais complexos serão mais caros do que os modelos mais simples.

Independentemente do caso, teremos uma chave de API (*API key*). Essa chave é utilizada para que os serviços de IA saibam de quem cobrar as mensagens, bem como para controlar acessos aos serviços de IA.

FIQUE ATENTO

A API key é como uma senha.

Nunca compartilhe, **nunca** commit no Git, **nunca** cole em um chat.

Se vazar, você paga a conta! Além disso, ao usar um serviço pago, procure implementar alguma lógica de restrição no uso de mensagens. Por exemplo: faz sentido termos usuários perguntando algo para a sua solução de IA ao enviar uma mensagem a cada 15 ou 30 segundos, certo? Mas será que faz sentido termos usuários enviando dezenas de mensagens dentro de um único minuto? Ou, será que faria sentido um único usuário envie centenas de mensagens bem grandes para uma IA?

E como seria testarmos uma IA paga em vez de uma local? Que tal vermos juntos nesta videoaula?

Para visualizar o vídeo, aponte a câmera do seu celular

System prompts

O ***system prompt*** (*prompt* de sistema) é uma das ferramentas mais poderosas para controlar o comportamento de uma LLM. Pense nele como as instruções de personalidade do seu assistente.

Exemplos de *system prompts*:

#CODIFICA

```
# Assistente formal
system_formal = """Você é um assistente profissional de uma empresa de advocacia.
Sempre use linguagem formal e técnica. Não use gírias ou expressões coloquiais.
Se não souber a resposta, diga claramente que não sabe."""[
```

#CODIFICA

```
# Assistente descontraído
system_casual = """Você é um amigo que ajuda com tecnologia.
Use linguagem informal, emojis são bem-vindos! 😊
Explique conceitos técnicos de forma simples, usando analogias do dia a dia."""[
```

#CODIFICA

```
# Assistente especializado
system_vendas = """Você é um especialista em vendas da empresa XYZ.
Seus produtos são: [lista de produtos].
Sempre tente entender a necessidade do cliente antes de sugerir um produto.
Nunca fale mal de concorrentes. Se não tiver o produto, sugira alternativas."""[
```

O *system prompt* é enviado como a primeira mensagem da conversa, com um papel (ou *role*) chamado "*system*". O modelo "lembra" dessas instruções de sistema ao longo de toda a conversa.

#FICADICA

Em entrevistas técnicas, é comum perguntarem "como você customizaria o comportamento de uma LLM para um caso de uso específico?"

Começar a resposta mencionando system prompts é um bom ponto de partida.

Exemplos de *prompts* por área

Os *system prompts* permitem aos nossos agentes entenderem melhor o contexto do problema que estamos tentando resolver.

Pensemos juntos em alguns exemplos:

Para profissionais de saúde

Aqui, perceba que temos dois focos específicos: em primeiro lugar, não se emite um diagnóstico (isto é, de responsabilidade do médico). Em segundo lugar, há uma preocupação extra com a acolhida e empatia considerando o contexto próprio da saúde.

#CODIFICA

```
 system_medico = """Você é um assistente de triagem médica para uma
clínica.
Seu papel é coletar informações iniciais do paciente de forma
acolhedora.
IMPORTANTE:
- NUNCA faça diagnósticos. Apenas colete sintomas.
- Se o paciente relatar sintomas graves (dor no peito, dificuldade
para respirar),
  oriente a buscar emergência IMEDIATAMENTE.
- Ao final, gere um resumo para o médico revisar.
Perguntas que você deve fazer:
1. Qual o motivo da consulta?
2. Há quanto tempo sente esses sintomas?
3. Está tomando alguma medicação?"""[
```

Para profissionais de direito

Aqui, há uma certa similaridade com o *prompt* dos profissionais de saúde: também não devemos emitir aconselhamento jurídico específico. Por outro lado, na área do direito, existe uma preocupação específica com o amparo legal (isto é, qual lei ou artigo sustenta uma determinada afirmação).

Naturalmente, o tom de voz depende do público-alvo. Aqui, há um pedido explícito por uma linguagem acessível porque o público-alvo poderia ser de pessoas de fora da área do direito. Se o público-alvo fosse de advogados teríamos um *prompt* diferente e, para juízes, um outro *prompt* específico etc.

#CODIFICA

```
system_juridico = """Você é um assistente jurídico especializado em
Direito do Consumidor.
Você ajuda clientes a entenderem seus direitos básicos.
IMPORTANTE:
- NUNCA forneça aconselhamento jurídico específico para casos
concretos.
- Sempre recomende consultar um advogado para casos complexos.
- Cite artigos do CDC quando relevante.
- Use linguagem acessível, evitando jargão jurídico."""[
```

Para profissionais de finanças

Em finanças, há um foco específico para o futuro: afinal, muitas discussões são voltadas a situações que são de médio e longo prazo, como investimentos e empréstimos. Também temos um foco específico em riscos, uma vez que há uma possibilidade iminente de ganho ou perda de dinheiro.

#CODIFICA

```
system_financeiro = """Você é um assistente de análise financeira.
Ajuda a interpretar demonstrativos e identificar pontos de atenção.
IMPORTANTE:
- Foque educação financeira, não em recomendações de investimento.
- Quando analisar números, sempre peça confirmação.
- Destaque riscos e pontos de atenção de forma clara.
- Seja conservador em projeções."""[
```

Resumindo ... ⬇️

Área	Foco do *system prompt*	Cuidado principal
Saúde	Coleta de informações, triage.	Nunca diagnosticar.
Direito	Educação jurídica.	Nunca aconselhar casos específicos.
Finanças	Análise e educação.	Nunca recomendar investimentos.
Vendas	Persuasão e informação.	Nunca mentir sobre produtos.
Educação	Explicação e paciência.	Adaptar ao nível do aluno.

O autor.

PRA SABER MAIS

Cada empresa possui orientações específicas para a criação de prompts. Dito isso, recomendo que dê uma olhada nas recomendações de prompt da Anthropic.

Gestão de contexto

Pense assim: quando você está numa conversa longa com um amigo, às vezes precisa lembrar o que foi dito no início. Se a conversa durar horas, você provavelmente vai esquecer alguns detalhes do começo, certo? Com LLMs, o problema é bem parecido.

A janela de contexto funciona como uma **memória de trabalho** do modelo, e ela é um espaço disponível para armazenar todas as mensagens da conversa atual: o *system prompt*, as perguntas do usuário, as respostas anteriores do assistente, e a resposta que ele está gerando agora. Tudo isso junto precisa caber na janela.

FIQUE POR DENTRO

A janela de contexto inclui tanto os **tokens de entrada** (input) quanto os **tokens de saída** (output). Ou seja, não adianta mandar um texto enorme achando que "sobra espaço", já que a resposta do modelo também consome tokens da mesma janela.

LLMs têm memória limitada, medida em ***tokens***. Um *token* é aproximadamente quatro caracteres em inglês ou três caracteres em português. A cada turno de conversa (isto é, uma pergunta e uma resposta) temos:

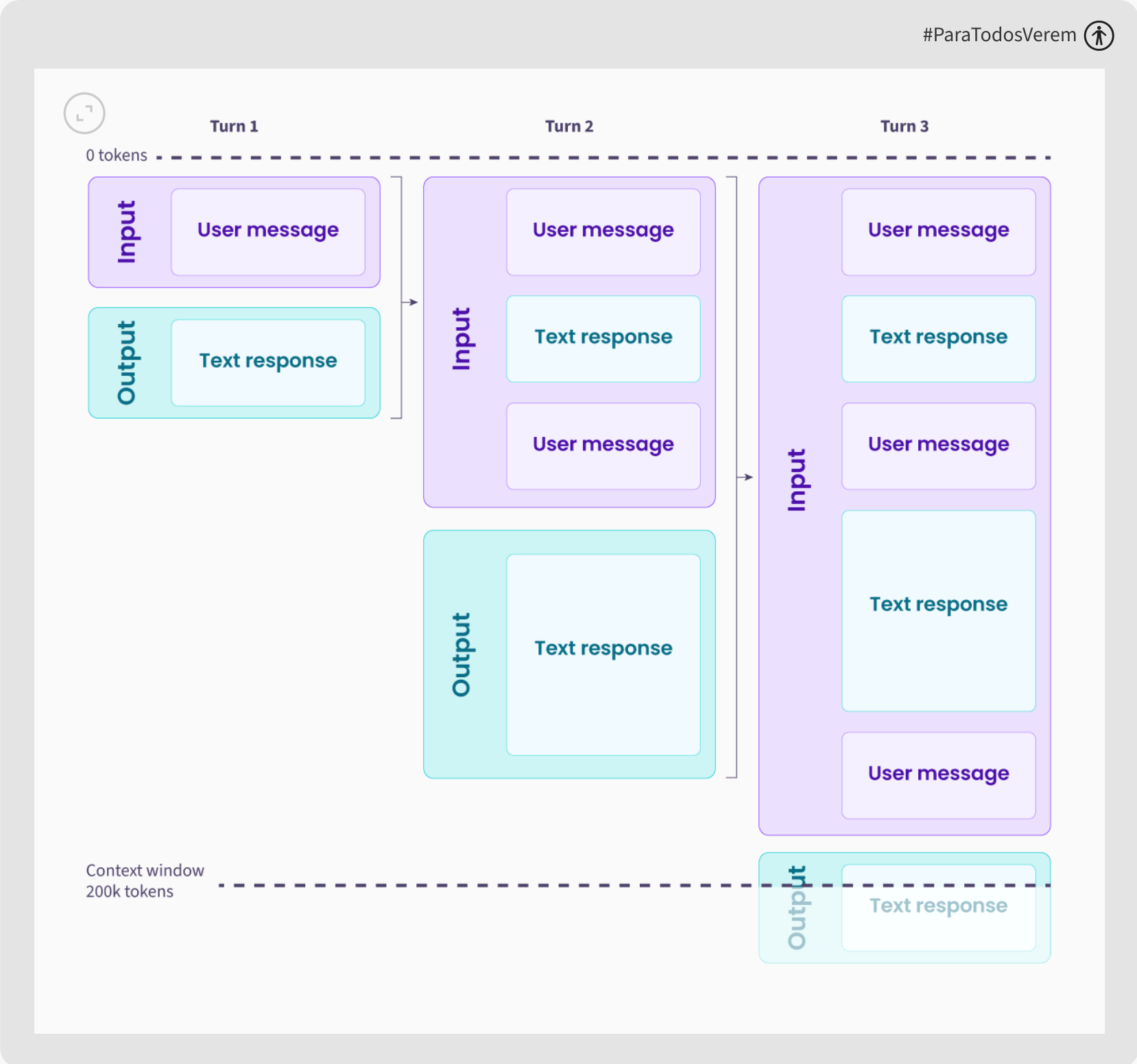
1

Entrada (input): iInclui todo o histórico anterior + a nova mensagem do usuário.

2

Saída (***output***): o modelo gera uma resposta que será incluída no histórico futuro.

Na imagem abaixo temos um exemplo de três turnos:



Fonte: o autor.

Observe que o crescimento é linear. Isso significa que cada mensagem nova se soma às anteriores. Se você não gerenciar isso, eventualmente vai estourar o limite e receber um erro.

Alguns modelos conseguem contornar isso: quando o total de tokens ultrapassa a janela, o sistema retorna um erro de validação em vez de simplesmente cortar o contexto silenciosamente. Isso é bom porque você sabe exatamente o que aconteceu, mas exige mais cuidado no gerenciamento.

Para você ter uma noção, veja qual é o tamanho da janela de contexto de alguns modelos de LLMs:

Modelo	Janela de contexto	Tipo	Observação
Llama 3.2 (Ollama)	8.192 tokens	Local/Gratuito	Bom para conversas curtas
Mistral (Ollama)	32.000 tokens	Local/Gratuito	Ótimo custo-benefício

Modelo	Janela de contexto	Tipo	Observação
GPT-3.5-turbo	16.385 tokens	API/Pago	Modelo econômico
GPT-4-turbo	128.000 tokens	API/Pago	Janela de contexto bem maior em relação ao modelo anterior
Claude Sonnet 4 / 4.5	200.000 tokens (padrão)	API/Pago	Pode chegar a **1 milhão** em beta!

Protegendo suas API *keys*

🚨 Este é um dos tópicos mais importantes para você compreender.

Chaves de API expostas estão entre os erros mais comuns e caros cometidos por desenvolvedores.

Vamos começar por um termo importante: *hardcoded*. *Hardcoded* significa que informações sensíveis estão escritas diretamente no código-fonte do projeto.

Veja estes exemplos problemáticos em Python:

#CODIFICA

PYTHON

```python
# ERRADO - API key hardcoded
client = OpenAI(api_key="sk-abc123xyz...")

# ERRADO - Em um arquivo Python público
API_KEY = "sk-abc123xyz..."[
```

Agora, qual é o problema nisso?

Quando expomos as chaves dessa forma, pessoas mal-intencionadas podem acessá-las e utilizá-las para outros fins. É como se deixássemos os dados de acesso ao banco escritos em um papel no bolso da calça: sempre existe o risco de perder e alguém ter acesso à sua conta, não é?

Para resolver esse problema e protegermos os nossos segredos (chaves de API, senhas, tokens), usamos **variáveis de ambiente** (*environment variables* ou *env vars*). Com elas, criamos um arquivo separado e protegido (como .env) que armazena essas informações sensíveis fora do código-fonte, mantendo-as seguras e longe de repositórios públicos.

#FICADICA

Na dúvida, use uma LLM para analisar o seu código com o objetivo de identificar segredos (secrets) e colocá-los em um arquivo .env próprio com este arquivo dentro de um .gitignore. Isto significa que queremos separar os secrets do seu projeto, e evitar que esses secrets sejam parte do seu repositório de código.

E mais: ⬇️

FIQUE ATENTO

Se você, acidentalmente, commitar uma API key, ela pode ser detectada por bots em segundos. Mesmo que você delete o commit, ela pode ter sido copiada. Nesse caso, revogue a chave imediatamente e gere uma nova.

© PUCPR - Todos os direitos reservados